

Getting started with Preact JS

A tiny React alternative

Arnab Mukherjee | arnab@digit.in

If you've used JavaScript UI frameworks then you might have observed that the framework is

mostly the largest part of your app's JavaScript size. In case of a particularly popular UI framework, React, this led to a bunch of developers into creating a lightweight alternative that ships with the same capabilities as React yet lets your code be the biggest part of the app.

INTRODUCTION

Preact is a 3kB alternative to React developed by Jason Miller, a developer at Google, and a bunch other contributors. Its goal was to provide the same API and features that React comes with, while being lightweight and super-fast!

It succeeds on both counts. Its speed is not only due to its small size - it is one of the fastest virtual DOM libraries out there, due to its simple and predictable diff implementation. In fact, Preact's goals can be put across in five points:

- **Performance:** Render quickly & efficiently
- **Size:** Small size, lightweight
- **Efficiency:** Effective memory usage
- **Understandability:** Understanding the codebase should take no more than a few hours
- **Compatibility:** Preact aims to be largely compatible with the React API.

All the above metrics, combined with Preact's wide support for brows-

ers has made it very popular among the developer community. Even companies like Housing.com, Uber and Lyft have used Preact to build mobile versions of their offerings at smaller sizes and improved performance.



PREACT VS REACT

A lot of developers swear by React for interface needs. Hence, it is pretty natural to wonder why Preact is even needed. There are a few factors that make its case.

File size

You cannot ignore the size difference between the two. React is at 45kB whereas Preact is, as we stated earlier, at 3kB. This is because Preact only keeps the essentials of React, and ditches the rest to remain lightweight.

class vs classname

In React, adding a class works something like this:

```
# React code
<div className="class-
here">Content Here</div>
```

While Preact supports `classname`, you can just use `class` for CSS classes.

```
# Preact code
<div class="class-
here">Content Here</div>
```

render()

For Preact, `this.props` and `this.state` are passed to `render()`. To understand the difference, take a look at the code here: <https://digit.in/renderPR>. As it is evident, the code looks cleaner in Preact with the removal of `this`.

You can even take it one notch higher and use destructuring as shown here: <https://digit.in/PRDestruc>.

Removal of PropTypes

Since PropTypes aren't exactly widely used, Preact has ditched them. If you still want to use PropTypes in your development, you can do so through `preact-compat` or manually.

Removal of Children

Children are not necessary in Preact since `props.children` is always an array. `React.children` is supported in `preact-compat`.

Lack of support for Synthetic events

This has been done since Preact's target for browser support is not along the same lines of this extra overhead. Instead, Preact puts to use the browser's native `addEventListener` for event handling. Checkout GlobalEventHandlers (<https://digit.in/GEHDOM>) for a complete list of DOM event handlers.

Also, a complete implementation of events would eventually lead to more maintenance and performance concerns and a larger API, which is directly against Preact's original goal.

Component

In Preact, components are an extendable class for building self updating, encapsulated pieces of User Interface with support for lifecycle methods. This is similar to `React.Component`. Check out the code here to understand it better: <https://digit.in/compPR>.

COMPATIBILITY WITH REACT

To ensure compatibility with React, Preact has a package called `preact-compat` which is intended to function as a compatibility layer between Preact